

Module 1

Tyk Streams

Managing Event-Driven APIs with Tyk

Agend

Foundational

01

Introduction

Understanding Streaming APIs and Why Tyk Streams

02

Architecture

Core Architecture and Flow

03

Hands-On

Setting up streaming API on Tyk

04

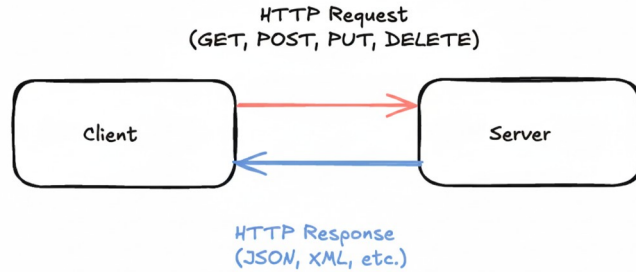
Advanced Use Cases

Beyond the core features

What are Streaming APIs

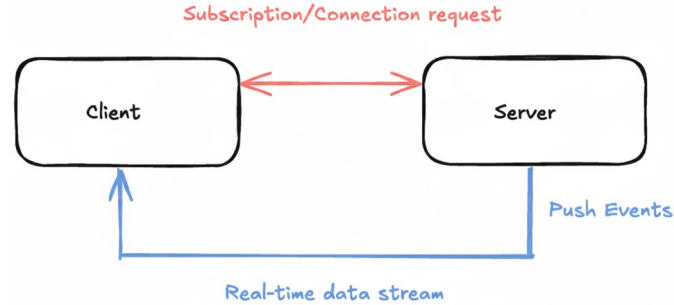
- Real-time, continuous data exchange
 - Server push vs client pull
 - No repeated TLS handshakes or re-auth overhead
- Client keeps connection open
 - Server maintains session state until closed
- Common patterns: SSE, WebSockets, Webhooks
- Enables low-latency, bidirectional communication
- Foundation for **event-driven architectures**

Traditional REST



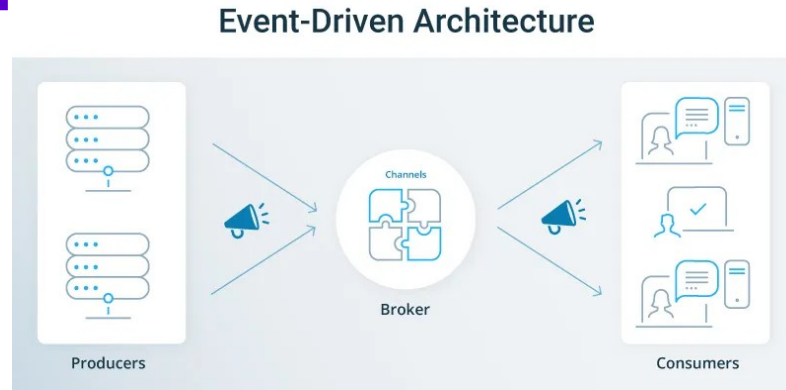
- **Client-Server:** The client initiates communication by sending an HTTP request to the server.
- **Request-Response:** Server processes request and sends a HTTP response back to the client
- **Stateless:** Each request from the client to the server must contain all the information needed to understand the request, as the server does not store any client context between requests.
- **On-demand:** Data is retrieved or manipulated only when explicitly requested by the client.

Streaming API



- **Client-Server:** The client establishes a persistent connection with the server.
- **Subscription/Connection:** The client typically subscribes to a specific data stream or event topic.
- **Push-based:** The server pushes real-time data or events to the client as they occur
- **Event-driven:** Data delivered in a continuous stream of events, suitable for applications requiring immediate updates.
- **Stateful (often):** Streaming APIs often maintain a stateful connection to manage the data stream and subscriptions.

Event-Driven Architecture Explained



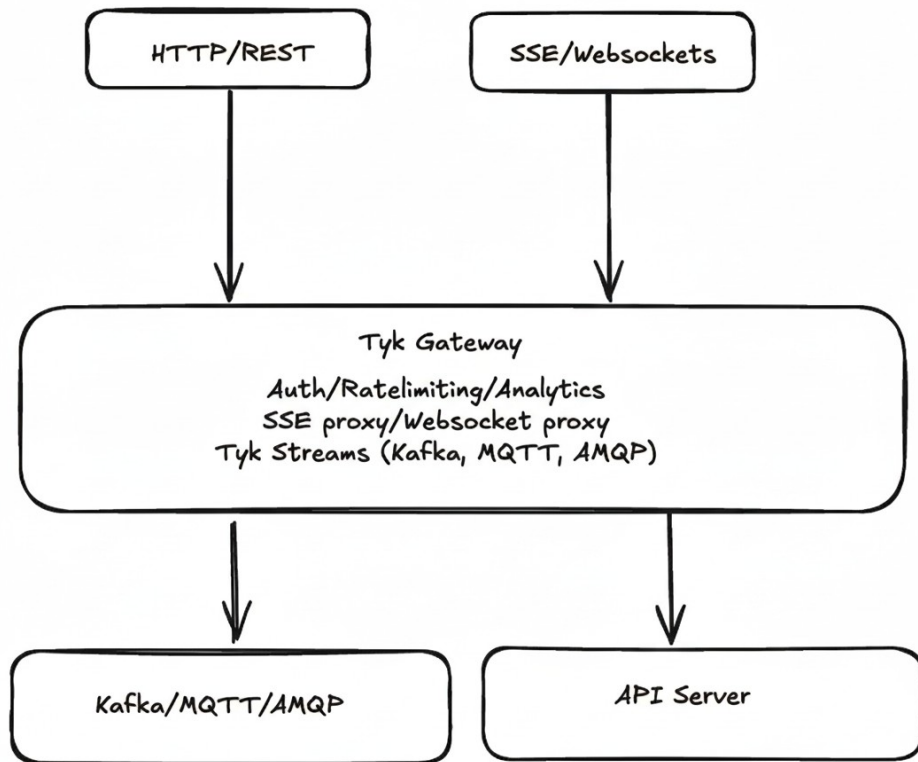
- Producers emit events, consumers react
- Asynchronous, decoupled microservices
- Common brokers: Kafka, MQTT, AMQP
- Improves scalability and resilience
- Uses own protocol - Kafka binary protocol, AMQP 0.9.1/1.0 etc

Tyk and Streaming APIs

Server-Sent Events (SSE)

- SSE Proxying: Native support for server-push over HTTP.
 - API Management Controls: Auth, rate limits, analytics
 - Ideal for: Notifications, live feeds, one-way updates.
-
- WebSockets
 - Proxying: Persistent, bidirectional channels managed by Tyk.
 - Full-duplex Communication: Chat, collaboration, dashboards.
 - Governance: Enforced with same policies as REST APIs.
-
- Tyk Streams for Event Brokers
 - Out-of-the-box Connectors: Kafka, MQTT, AMQP
 - Protocol Mediation: Bridge HTTP/WS/SSE ↔ Brokers.
 - Examples: Expose Kafka topics as WebSockets, convert MQTT to HTTP webhooks

Tyk and Streaming APIs



Why Tyk Streams?

- Same Tyk security, rate limits, analytics
 - Authentication
 - Policies
- Connect REST ↔ Kafka seamlessly
 - REST producers that publish events via HTTP POST
 - to Kafka, AMQP, MQTT brokers that handle distribution
- No external broker configuration in Gateway
- Unified lifecycle management
 - Versioning
 - Automation
 - Governance

Core Concept: Unified Streaming Layer

Powered by BENTO Stream Processor

- Event-processing engine embedded in the Gateway
- Supports Bloblang transformations (map, filter, enrich)
- Optional branching, merging, rate control, error handling
- Unsafe processors (file, subprocess) removed unless explicitly enabled
- High-throughput, memory-buffered streaming for async workloads

Defined in x-tyk-streaming (OAS Only)

- Only Tyk OAS-based APIs support Streams (not Tyk Classic)
- Central place to define inputs, processors, outputs, topics, paths
- Declarative configuration via YAML/JSON → perfect for GitOps
- Works with Dashboard UI + Tyk Sync automation
- Can define multiple named streams inside one API definition

Core Concept: Unified Streaming Layer

INPUTS → PROCESSORS → OUTPUTS Flow Model

- Inputs
 - Accept events via HTTP, WebSocket, Kafka, MQTT, AMQP
 - Acts as producer side: Tyk ingests data via REST, WebSockets, or brokers
- Processors
 - Transform or filter messages with Bloblang
 - Enrich events via HTTP lookups
 - Add metadata, timestamps, or normalize schema
 - Validate payloads, reshape event structure
- Outputs
 - Publish to Kafka, MQTT, AMQP
 - Expose live streams via WebSocket Server or SSE
 - Execute webhooks, push to Developer Portal (portal_webhook)
 - Fan-out to multiple outputs in parallel

Capabilities Overview

Protocol Mediation: HTTP ↔ Kafka, SSE, MQTT, AMQP

- Bridge synchronous (REST) and asynchronous (broker) systems
- Accept events via HTTP or WebSocket, publish to Kafka/MQTT/AMQP
- Consume broker events, stream out via WebSocket/SSE
- Enables REST-based producers & real-time consumers without custom services

Security via Existing Middleware Chain

- Inherits AuthN: API Keys, JWT, OAuth 2.0, mTLS
- Applies Rate Limits, Quotas, Policies to stream endpoints
- Enforces RBAC, org separation, tagging & ACLs
- Same governance stack used for REST + GraphQL APIs

Transformations with Bloblang/Bento

- Map, filter, reshape payloads inline
- Add metadata: timestamps, org IDs, device info
- Enrich via external HTTP lookups
- Implement branching, fan-out, error correction patterns

Capabilities Overview

Developer Portal via portal_webhook

- Push events to Portal-registered webhooks
- Monetize streaming APIs (paid subscriptions to WebSocket/SSE topics)
- Automate webhook credential retrieval
- Integrate event flows with external partner systems

Analytics via Pump Exporters

- Streams feed into existing analytics recorder
- Export via Pump to Postgres, Prometheus, Datadog, Elastic
- Track throughput, latency, connection counts, errors
- Filter analytics by API, org, or event type

Architecture & Flow

Components: Middleware → Manager → Instance → Analytics

- Middleware: entry point, validates path & auth
- Manager: builds or reuses Bento stream instances
- Instance: executes Input → Processor → Output pipeline
- Analytics: records per-event metrics for Pump

Stream Manager Caches Stream Configurations

- Caches by configuration hash for reuse
- Garbage collects idle stream instances
- Tracks active subscriptions (SSE/WS)

Registers Default Paths (depending on connectors)

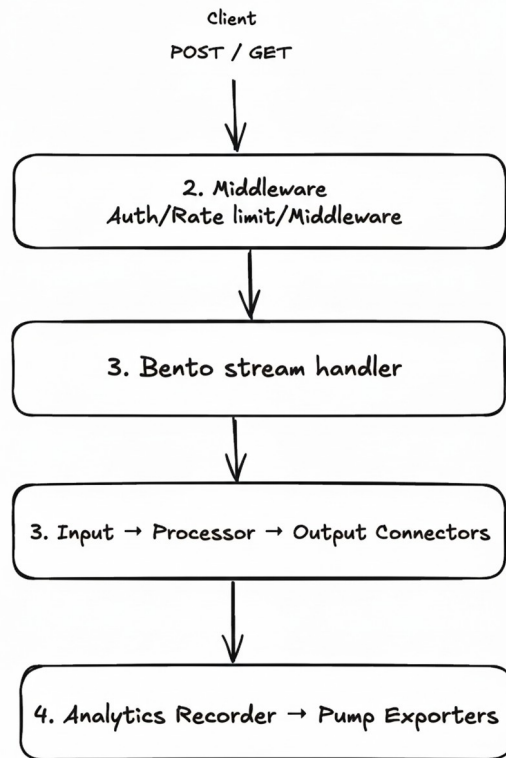
- /post → HTTP ingestion endpoint
- /post/ws → WebSocket ingestion
- /get/stream → SSE/WebSocket delivery
- Automatically wires paths to the right stream

Collects Metrics for Pump

- Captures event counts, connection duration, failures
- Exports via Pump to Postgres, Prometheus, Datadog, Elastic
- Same analytics pipeline as REST/GraphQL APIs

Request Lifecycle

1. Client sends HTTP POST / GET
2. Middleware validates & authenticates
3. Routed to Bento handler
4. Processed through input/output connectors
5. Analytics logged



Hands-On Workshop

Setting up and proxying a streaming API on Tyk

Example Scenario Overview

Tyk Streams: End-to-End Async Messaging via HTTP → Kafka → Browser

We will demonstrate how Tyk Streams can:

- Accept HTTP POST requests from a browser
- Convert them into Kafka messages
- Consume Kafka responses
- Push results back to the browser using SSE/WebSockets

Use case:

User types “Tell me a joke” → Backend responds with a joke → Delivered in real-time.

Tyk as an API Producer Gateway (Client → Stream)

Goal: Accept browser input and publish into Kafka.

Flow

1. Browser sends POST /chat → Tyk Gateway
2. A Stream-enabled API definition processes the payload
3. Tyk publishes the message to Kafka topic (e.g., chat)
4. Backend “Joker Service” consumes from that topic

Value

1. Protocol bridging: HTTP → Kafka
2. Security: Tyk enforces auth, quotas, rate limits before Kafka
3. Decoupling: Browser never touches Kafka directly

Tyk as an API Consumer Gateway (Stream → Client)

Goal: Deliver async Kafka responses to the browser.

Flow

1. Joker Service publishes joke to Kafka topic discussion
2. Tyk Streams Consumer subscribes to discussion
3. Tyk pushes event to client using SSE or WebSockets
4. Client sees joke appear instantly

Value

- Kafka → SSE bridging
- Client stays HTTP-native (no Kafka client needed)
- Backend stays Kafka-native (no WebSockets needed)

Prerequisites

To follow the live demo, you will need:

- Docker (run full Tyk stack locally)
- Git (clone repo)
- Dashboard license (for Streams API creation)
- curl + jq (for testing SSE + POST)

Step 1: Clone Demo Repository

```
git clone https://github.com/TykTechnologies/tyk-demo  
cd tyk-demo
```

Step 2: Enable Tyk Streams

Create .env file

```
DASHBOARD_LICENCE=<your-dashboard-license>  
GATEWAY_IMAGE_REPO=tyk-gateway-ee  
TYK_DB_STREAMING_ENABLED=true  
TYK_GW_STREAMING_ENABLED=true
```

Step 3: Start the Stack (Gateway + Dashboard + Kafka)

Run :

```
./up.sh
```

Script performs:

- Tyk Gateway (EE)
- Tyk Dashboard
- Kafka broker
- Required dependencies

Copy credentials shown at completion.

Step 4: Create Producer Stream API

1. Create `producer.json`

1. Run:

```
curl -H "Authorization: <your-api-key>" \  
  -H "Content-Type: application/vnd.tyk.streams.oas" \  
  http://localhost:3000/api/apis/streams \  
  -d @producer.json
```

This API receives POST `/chat` and publishes messages into Kafka.

Step 5: Create Consumer Stream API

1. Create `consumer.json`

1. Run:

```
curl -H "Authorization: <your-api-key>" \  
  -H "Content-Type: application/vnd.tyk.streams.oas" \  
  http://localhost:3000/api/apis/streams \  
  -d @consumer.json
```

This API subscribes to Kafka topic discussion and delivers messages using SSE.

Step 6: Start the Joker Service

Run:

```
chmod +x joker-service.sh
```

```
./joker-service.sh
```

The service:

- Listens to Kafka chat topic
- Generates jokes
- Publishes responses to discussion topic

Step 7: Test — Listen for SSE Events

Terminal 1:

```
curl -N http://tyk-gateway.localhost:8080/jippity-discuss/sse
```

Step 8: Test — Send Requests to Tyk Stream Producer

Terminal 2:

```
curl -X POST \  
  
http://tyk-gateway.localhost:8080/jippity-chat/chat \  
  
-H "Content-Type: text/plain" \  
  
-d "Tell me a joke."
```

Demo Summary

- HTTP → Kafka → HTTP/SSE achieved
- Fully decoupled architecture
- Browser needs only HTTP
- Backend needs only Kafka
- Tyk Streams acts as the universal bridge
- This pattern generalises to:
 - Chat systems
 - Event streaming APIs
 - Real-time dashboards
 - AI inference pipelines